In [7]:

```
# ==========================================
# GEE Python API: Decadal Urban Thermal Comfort Analysis
# Output: LST, UTFVI, UTCI (High Res), Trend Graphs, Tables
# ==========================================

import ee
import geemap
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import xarray as xr
import rioxarray

# ---------------------------------------------------------
# STEP 1: Initialization and Study Area Setup
# ---------------------------------------------------------
# ee.Authenticate() # Uncomment if executing for the very first time.
ee.Initialize()

def mask_clouds(image):
    qa = image.select('QA_PIXEL')
    cloud_shadow_bit = 1 << 4
    clouds_bit = 1 << 3
    mask = qa.bitwiseAnd(cloud_shadow_bit).eq(0).And(qa.bitwiseAnd(clouds_bit).eq(0))
    return image.updateMask(mask)

dhaka = ee.FeatureCollection("FAO/GAUL/2015/level2") \
    .filter(ee.Filter.eq('ADM1_NAME', 'Dhaka')) \
    .filter(ee.Filter.eq('ADM2_NAME', 'Dhaka'))

sample_points = ee.FeatureCollection.randomPoints(dhaka.geometry(), 500)

# ---------------------------------------------------------
# STEP 2: 2005 Processing & Exports
# ---------------------------------------------------------
print("Processing 2005 Data...")

def calc_lst_l5(image):
    st = image.select('ST_B6').multiply(0.00341802).add(149.0).subtract(273.15)
    return st.rename('LST')

l5_2005 = ee.ImageCollection("LANDSAT/LT05/C02/T1_L2") \
    .filterBounds(dhaka).filterDate('2005-03-01', '2005-05-31').map(mask_clouds)
lst_2005 = l5_2005.map(calc_lst_l5).mean().clip(dhaka)

mean_lst_2005 = ee.Number(lst_2005.reduceRegion(ee.Reducer.mean(), dhaka.geometry(), 30).get('LS
utfvi_2005 = lst_2005.subtract(mean_lst_2005).divide(mean_lst_2005).rename('UTFVI')

# Applied bilinear resampling to smooth the 27km ERA5 data into a continuous high-res gradient
era5_2005 = ee.ImageCollection("projects/climate-engine-pro/assets/ce-era5-heat") \
    .filterBounds(dhaka).filterDate('2005-03-01', '2005-05-31')
utci_2005 = era5_2005.select('utci_max').mean().subtract(273.15).resample('bilinear').clip(dhaka

img_2005 = lst_2005.addBands(utfvi_2005).addBands(utci_2005)

# Extract Data
sampled_dict_2005 = img_2005.sampleRegions(collection=sample_points, scale=30, geometries=False)
data_2005 = pd.DataFrame([feat['properties'] for feat in sampled_dict_2005['features']]).dropna(
data_2005.to_csv('Table_Dhaka_Data_2005.csv', index=False)
```

```
# Export Map TIFs (Scale set to 30m for maximum resolution)
geemap.ee_export_image(lst_2005.unmask(-9999), filename='Dhaka_LST_2005.tif', scale=30, region=dl
geemap.ee_export_image(utfvi_2005.unmask(-9999), filename='Dhaka_UTFVI_2005.tif', scale=30, regi
geemap.ee_export_image(utci_2005.unmask(-9999), filename='Dhaka_UTCI_2005.tif', scale=30, region=


# ---------------------------------------------------------
# STEP 3: 2015 Processing & Exports
# ---------------------------------------------------------
print("Processing 2015 Data...")

def calc_lst_l8(image):
    st = image.select('ST_B10').multiply(0.00341802).add(149.0).subtract(273.15)
    return st.rename('LST')

l8_2015 = ee.ImageCollection("LANDSAT/LC08/C02/T1_L2") \
    .filterBounds(dhaka).filterDate('2015-03-01', '2015-05-31').map(mask_clouds)
lst_2015 = l8_2015.map(calc_lst_l8).mean().clip(dhaka)

mean_lst_2015 = ee.Number(lst_2015.reduceRegion(ee.Reducer.mean(), dhaka.geometry(), 30).get('LS
utfvi_2015 = lst_2015.subtract(mean_lst_2015).divide(mean_lst_2015).rename('UTFVI')

era5_2015 = ee.ImageCollection("projects/climate-engine-pro/assets/ce-era5-heat") \
    .filterBounds(dhaka).filterDate('2015-03-01', '2015-05-31')
utci_2015 = era5_2015.select('utci_max').mean().subtract(273.15).resample('bilinear').clip(dhaka

img_2015 = lst_2015.addBands(utfvi_2015).addBands(utci_2015)

# Extract Data
sampled_dict_2015 = img_2015.sampleRegions(collection=sample_points, scale=30, geometries=False)
data_2015 = pd.DataFrame([feat['properties'] for feat in sampled_dict_2015['features']]).dropna(
data_2015.to_csv('Table_Dhaka_Data_2015.csv', index=False)

# Export Map TIFs
geemap.ee_export_image(lst_2015.unmask(-9999), filename='Dhaka_LST_2015.tif', scale=30, region=dl
geemap.ee_export_image(utfvi_2015.unmask(-9999), filename='Dhaka_UTFVI_2015.tif', scale=30, regi
geemap.ee_export_image(utci_2015.unmask(-9999), filename='Dhaka_UTCI_2015.tif', scale=30, region=


# ---------------------------------------------------------
# STEP 4: 2025 Processing & Exports
# ---------------------------------------------------------
print("Processing 2025 Data...")

l8_2025 = ee.ImageCollection("LANDSAT/LC08/C02/T1_L2") \
    .filterBounds(dhaka).filterDate('2025-03-01', '2025-05-31').map(mask_clouds)
lst_2025 = l8_2025.map(calc_lst_l8).mean().clip(dhaka)

mean_lst_2025 = ee.Number(lst_2025.reduceRegion(ee.Reducer.mean(), dhaka.geometry(), 30).get('LS
utfvi_2025 = lst_2025.subtract(mean_lst_2025).divide(mean_lst_2025).rename('UTFVI')

era5_2025 = ee.ImageCollection("projects/climate-engine-pro/assets/ce-era5-heat") \
    .filterBounds(dhaka).filterDate('2025-03-01', '2025-05-31')
utci_2025 = era5_2025.select('utci_max').mean().subtract(273.15).resample('bilinear').clip(dhaka

img_2025 = lst_2025.addBands(utfvi_2025).addBands(utci_2025)

# Extract Data
sampled_dict_2025 = img_2025.sampleRegions(collection=sample_points, scale=30, geometries=False)
data_2025 = pd.DataFrame([feat['properties'] for feat in sampled_dict_2025['features']]).dropna(
data_2025.to_csv('Table_Dhaka_Data_2025.csv', index=False)
```

```python
# Export Map TIFs
geemap.ee_export_image(lst_2025.unmask(-9999), filename='Dhaka_LST_2025.tif', scale=30, region=dl
geemap.ee_export_image(utfvi_2025.unmask(-9999), filename='Dhaka_UTFVI_2025.tif', scale=30, regi
geemap.ee_export_image(utci_2025.unmask(-9999), filename='Dhaka_UTCI_2025.tif', scale=30, region=


# ---------------------------------------------------------
# STEP 5: Generating Binned Trend Line Graphs
# ---------------------------------------------------------
print("Generating Binned Trend Line Graphs...")
datasets = {'2005': (data_2005, 'blue'), '2015': (data_2015, 'orange'), '2025': (data_2025, 'darl

for year, (df, color) in datasets.items():
    df_sorted = df.sort_values(by='UTFVI').copy()
    df_sorted['UTFVI_bins'] = pd.cut(df_sorted['UTFVI'], bins=15)
    trend = df_sorted.groupby('UTFVI_bins', observed=True)['UTCI'].mean().reset_index()
    trend['UTFVI_mid'] = trend['UTFVI_bins'].apply(lambda x: x.mid).astype(float)
    trend = trend.dropna()

    plt.figure(figsize=(8, 6))
    plt.plot(trend['UTFVI_mid'], trend['UTCI'], color=color, linewidth=2.5, marker='o',
             markersize=8, markerfacecolor='white', markeredgewidth=2)

    plt.xlabel('Urban Thermal Field Variance Index (UTFVI)', fontsize=12)
    plt.ylabel('Mean Universal Thermal Climate Index (°C)', fontsize=12)
    plt.title(f'{year}: Surface Degradation vs Human Heat Stress Trend', fontsize=14, fontweight=
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.savefig(f'Graph_LineTrend_{year}.jpg', dpi=300)
    plt.close()


# ---------------------------------------------------------
# STEP 6: Cartographic Rendering Function
# ---------------------------------------------------------
print("Generating High-Resolution Maps...")
years = ['2005', '2015', '2025']

# Reusable cartographic function to ensure perfect consistency
def render_map(filename, cmap, title, cbar_label, output_name):
    da = rioxarray.open_rasterio(filename).squeeze()
    da = da.where(da > -9000) # Mask NoData

    fig, ax = plt.subplots(figsize=(8, 8))
    v_min, v_max = da.quantile(0.02).values, da.quantile(0.98).values

    plot = da.plot(ax=ax, cmap=cmap, vmin=v_min, vmax=v_max, add_colorbar=False)

    cbar = fig.colorbar(plot, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label(cbar_label, rotation=270, labelpad=15, fontsize=12)

    ax.set_title(title, fontsize=16, fontweight='bold', pad=15)
    ax.set_xlabel('Longitude', fontsize=12)
    ax.set_ylabel('Latitude', fontsize=12)
    ax.set_facecolor('white')

    for spine in ax.spines.values():
        spine.set_edgecolor('black')
        spine.set_linewidth(2)

    # Corrected, Highly Visible North Arrow
```

```
    ax.annotate('N', xy=(0.08, 0.95), xytext=(0.08, 0.88),
                arrowprops=dict(facecolor='black', width=4, headwidth=12),
                ha='center', va='center', fontsize=16, fontweight='bold',
                xycoords=ax.transAxes, zorder=10,
                bbox=dict(boxstyle="round,pad=0.3", facecolor="white", edgecolor="none", alpha=0

    plt.tight_layout()
    plt.savefig(output_name, dpi=300, bbox_inches='tight')
    plt.close()
    print(f"Exported: {output_name}")

# Generate all maps using the standardized function
for year in years:
    # 1. LST Maps
    render_map(f'Dhaka_LST_{year}.tif', 'RdYlBu_r',
               f'Spatial Distribution of LST\nDhaka City ({year})',
               'Land Surface Temperature (°C)', f'Map_Standalone_LST_{year}.jpg')

    # 2. UTFVI Maps
    render_map(f'Dhaka_UTFVI_{year}.tif', 'inferno',
               f'Spatial Distribution of UTFVI\nDhaka City ({year})',
               'Urban Thermal Field Variance Index (UTFVI)', f'Map_Standalone_UTFVI_{year}.jpg')

    # 3. UTCI Maps
    render_map(f'Dhaka_UTCI_{year}.tif', 'Spectral_r',
               f'Spatial Distribution of Human Heat Stress (UTCI)\nDhaka City ({year})',
               'Universal Thermal Climate Index (°C)', f'Map_Standalone_UTCI_{year}.jpg')

print("All advanced standalone files successfully exported.")
```

```
Processing 2005 Data...
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/9bbf
31acf8a1cf31f4059a1798f01df4-8df99dc5527adb15fa78c9bc76958db4:getPixels
Please wait ...
Data downloaded to I:\Dhaka_LST_2005.tif
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/67b4
355a1ceb2358137054b9d256cb20-3c2e202f14d73b0ee1f81e3a1b98ef04:getPixels
Please wait ...
Data downloaded to I:\Dhaka_UTFVI_2005.tif
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/b17f
99b0bae3f0490ef19ced2d56d7ee-1739ae9994691abdfb602bb8edb02d8d:getPixels
Please wait ...
Data downloaded to I:\Dhaka_UTCI_2005.tif
Processing 2015 Data...
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/695a
da0ce8cac57a9d1d4b6cc6da6532-8e2fff177d65ccfdc483cab06c209d5e:getPixels
Please wait ...
Data downloaded to I:\Dhaka_LST_2015.tif
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/7c65
62352547368d5b4fb137ca04f435-e0a26d32b2257fec8a16d9132f401cf1:getPixels
Please wait ...
Data downloaded to I:\Dhaka_UTFVI_2015.tif
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/1b18
8611a574c4bc83d53072e691bc3f-e8792bdd1ae6dd2ce7b8afe7a241bcb3:getPixels
Please wait ...
Data downloaded to I:\Dhaka_UTCI_2015.tif
Processing 2025 Data...
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/d940
586c84d7fd57703461c4c87e91b4-44ca81715f6ee3fb77871c15fd5f869c:getPixels
Please wait ...
Data downloaded to I:\Dhaka_LST_2025.tif
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/ae07
74f5d9468497b47bdd6019cdf0e1-34779478cdb368142d7761b437f691b1:getPixels
Please wait ...
Data downloaded to I:\Dhaka_UTFVI_2025.tif
Generating URL ...
Downloading data from https://earthengine.googleapis.com/v1/projects/489792775305/thumbnails/4e21
f61447f74020c0aa22a50970a6b6-1cc82d28846d48b5a15f6bb22da97217:getPixels
Please wait ...
Data downloaded to I:\Dhaka_UTCI_2025.tif
Generating Binned Trend Line Graphs...
Generating High-Resolution Maps...
Exported: Map_Standalone_LST_2005.jpg
Exported: Map_Standalone_UTFVI_2005.jpg
Exported: Map_Standalone_UTCI_2005.jpg
Exported: Map_Standalone_LST_2015.jpg
Exported: Map_Standalone_UTFVI_2015.jpg
Exported: Map_Standalone_UTCI_2015.jpg
Exported: Map_Standalone_LST_2025.jpg
Exported: Map_Standalone_UTFVI_2025.jpg
Exported: Map_Standalone_UTCI_2025.jpg
All advanced standalone files successfully exported.
```

In [13]:

```
# ==========================================
# GEE Python API: Master 4x3 Composite Grid
# Memory-Safe Version (Using imshow)
# ==========================================

import pandas as pd
import matplotlib.pyplot as plt
import rioxarray

print("Generating Master 4x3 Publication Grid...")

# Set up the 4x3 grid (15x20 is large enough for high quality, but safe for memory)
fig, axes = plt.subplots(nrows=4, ncols=3, figsize=(15, 20))
years = ['2005', '2015', '2025']
colors = ['blue', 'orange', 'darkred']

for i, year in enumerate(years):

    # ---------------------------------------------------
    # ROW 1: LST Maps
    # ---------------------------------------------------
    da_lst = rioxarray.open_rasterio(f'Dhaka_LST_{year}.tif').squeeze()
    da_lst = da_lst.where(da_lst > -9000)

    v_min, v_max = float(da_lst.quantile(0.02)), float(da_lst.quantile(0.98))
    # THE MEMORY FIX: Using .plot.imshow() instead of .plot()
    plot_lst = da_lst.plot.imshow(ax=axes[0, i], cmap='RdYlBu_r', vmin=v_min, vmax=v_max, add_co

    cbar = fig.colorbar(plot_lst, ax=axes[0, i], fraction=0.046, pad=0.04)
    cbar.set_label('Land Surface Temp (°C)', rotation=270, labelpad=15)
    axes[0, i].set_title(f'LST ({year})', fontweight='bold')

    # ---------------------------------------------------
    # ROW 2: UTFVI Maps
    # ---------------------------------------------------
    da_utfvi = rioxarray.open_rasterio(f'Dhaka_UTFVI_{year}.tif').squeeze()
    da_utfvi = da_utfvi.where(da_utfvi > -9000)

    v_min, v_max = float(da_utfvi.quantile(0.02)), float(da_utfvi.quantile(0.98))
    plot_utfvi = da_utfvi.plot.imshow(ax=axes[1, i], cmap='inferno', vmin=v_min, vmax=v_max, add_

    cbar = fig.colorbar(plot_utfvi, ax=axes[1, i], fraction=0.046, pad=0.04)
    cbar.set_label('UTFVI', rotation=270, labelpad=15)
    axes[1, i].set_title(f'UTFVI ({year})', fontweight='bold')

    # ---------------------------------------------------
    # ROW 3: UTCI Maps
    # ---------------------------------------------------
    da_utci = rioxarray.open_rasterio(f'Dhaka_UTCI_{year}.tif').squeeze()
    da_utci = da_utci.where(da_utci > -9000)

    v_min, v_max = float(da_utci.quantile(0.02)), float(da_utci.quantile(0.98))
    plot_utci = da_utci.plot.imshow(ax=axes[2, i], cmap='Spectral_r', vmin=v_min, vmax=v_max, ad

    cbar = fig.colorbar(plot_utci, ax=axes[2, i], fraction=0.046, pad=0.04)
    cbar.set_label('UTCI (°C)', rotation=270, labelpad=15)
    axes[2, i].set_title(f'UTCI ({year})', fontweight='bold')

    # ---------------------------------------------------
    # ROW 4: Correlation Scatter Plots
    # ---------------------------------------------------
```

```python
    df = pd.read_csv(f'Table_Dhaka_Data_{year}.csv')
    axes[3, i].scatter(df['UTFVI'], df['UTCI'], alpha=0.5, color=colors[i], edgecolor='black', s

    axes[3, i].set_xlabel('UTFVI', fontsize=11)
    axes[3, i].set_ylabel('UTCI (°C)', fontsize=11)
    axes[3, i].set_title(f'Correlation ({year})', fontweight='bold')
    axes[3, i].grid(True, linestyle='--', alpha=0.6)

    axes[3, i].set_xlim([df['UTFVI'].min() - 0.02, df['UTFVI'].max() + 0.02])
    axes[3, i].set_ylim([df['UTCI'].min() - 0.5, df['UTCI'].max() + 0.5])

    # ---------------------------------------------------
    # CARTOGRAPHIC ELEMENTS (Apply to Maps Only)
    # ---------------------------------------------------
    for row in [0, 1, 2]:
        axes[row, i].set_xlabel('Longitude', fontsize=9)
        axes[row, i].set_ylabel('Latitude', fontsize=9)
        axes[row, i].set_facecolor('white')

        # Simple, solid North Arrow
        axes[row, i].annotate('N', xy=(0.08, 0.95), xytext=(0.08, 0.88),
                              arrowprops=dict(facecolor='black', width=3, headwidth=10),
                              ha='center', va='center', fontsize=12, fontweight='bold',
                              xycoords=axes[row, i].transAxes, zorder=10,
                              bbox=dict(boxstyle="round,pad=0.2", facecolor="white", edgecolor="

# Apply simple black borders (neatlines) to all graphs and maps
for row in range(4):
    for col in range(3):
        for spine in axes[row, col].spines.values():
            spine.set_edgecolor('black')
            spine.set_linewidth(1.2)

# Prevent overlap and save
plt.tight_layout(pad=2.0)
plt.savefig('Master_Grid_4x3_Dhaka.jpg', dpi=300, bbox_inches='tight')
plt.close()

print("Master Grid successfully generated and saved!")
```

```
Generating Master 4x3 Publication Grid...
Master Grid successfully generated and saved!
```

In [ ]: